2.g Types — Arrays, Type Checks, Casts & Aliases

Working with Arrays and Kotlin’s Type System

Contents

1 **Arrays in Kotlin**
Creating, accessing, and transforming arrays with arrayOf, Array constructor & primitives

2 **Type Checks & Smart Casts**
is / !is operators, smart casting, and when expressions

3 **Explicit Casts & Type Aliases**
as / as? cast operators, and typealias for readable type names

4 **Practice Exercises**
Solved and unsolved exercises combining arrays, casts & aliases

Arrays in Kotlin

An array is a fixed-size, mutable collection that stores elements of the same type. Represented by the Array class. Arrays are invariant — Array<String> cannot be assigned to Array<Any>.

3 ways to create arrays:

arrayOf() Function

Pass values directly:

```
val nums = arrayOf(1, 2, 3)
val names = arrayOf<String>(
  "Alice", "Bob")
```

Also: emptyArray(), arrayOfNulls()

Array Constructor

Size + init lambda:

```
val squares = Array(5) { i ->
  i * i
}
// [0, 1, 4, 9, 16]
```

Primitive Type Arrays

No boxing overhead:
IntArray, DoubleArray, BooleanArray, CharArray...

```
val ints = intArrayOf(1, 2, 3)
val dbls = DoubleArray(5) { 0.0 }
```

Key Facts

- Arrays have **fixed size** — you cannot add or remove elements after creation
- Elements are **mutable** — values can be changed via index or set()
- For most use cases, prefer **collections** (List, Set, Map) over arrays

Array Operations

Access, iterate, compare & transform arrays

```
fun main() {
    val fruits = arrayOf("Apple", "Banana",
"Cherry")

    // Access by index
    println(fruits[0])  // Apple
    fruits[0] = "Avocado"

    // Iterate
    for (fruit in fruits) {
        println(fruit)
    }

    // Spread operator + vararg
    val nums = intArrayOf(1, 2, 3)
    println(nums.sum())   // 6
}
```

Key Operations

- Access: **array[i]** or **array.get(i)**
- Modify: **array[i] = value** or **.set(i, v)**
- Size: **.size** property (fixed)
- Contains: **"x" in array**
- Spread: ***array** passes each element to vararg

Useful Functions

- .sum(), .shuffle(), .sorted()
- .toList(), .toSet(), .toTypedArray()
- .contentEquals() for comparison

Type Checks: is & !is

Use the is operator (or !is to negate) to check if an object conforms to a given type at runtime.

```kotlin
fun describe(obj: Any): String =
    when (obj) {
        is Int    -> "Int: $obj"
        is String -> "String: len=${obj.length}"
        is Double -> "Double: $obj"
        else      -> "Unknown"
    }

fun main() {
    println(describe("Hello"))
    // String: len=5
    println(describe(42))
    // Int: 42
}
```

Type Checks

- **is** checks if an object is a given type
- **!is** is the negated form
- Works in **if**, **when**, and **while** expressions

Smart Casts

- After an **is** check, Kotlin **automatically casts** the variable to the checked type
- No explicit cast needed — access type-specific members directly
- Works with **val** properties and local **var** (if not modified)
- **Never** works with var class properties

kotlinlang.org/docs/typecasts

Explicit Casts: as & as?

Kotlin has two cast operators: as (unsafe) throws ClassCastException on failure, as? (safe) returns null instead.

Unsafe Cast: as

- Throws **ClassCastException** if cast fails
- null cannot be cast to non-null types

```
val x: String = obj as String
// Crashes if obj is not String!
```

Safe Cast: as?

- Returns **null** if cast fails (no exception)
- Result type is always nullable

```
val x: String? = obj as? String
// x is null if obj is not String
```

Best Practices

- Prefer **smart casts** (is check) over explicit casts when possible — they are safer and more idiomatic
- Use **as?** when the cast might fail and you want to handle null gracefully
- Use **as** only when you are **certain** the cast will succeed

Type Aliases

Type aliases provide alternative names for existing types. They do not introduce new types — they are expanded to the underlying type at compile time.

```kotlin
// Shorten complex types
typealias NodeSet = Set<Network.Node>

// Alias for function types
typealias Predicate<T> = (T) -> Boolean

// Alias for inner classes
typealias MyHandler = (Int, String, Any) -> Unit

fun foo(p: Predicate<Int>) = p(42)

fun main() {
    println(foo { it > 0 })  // true
}
```

Key Points

- Syntax: **`typealias Name = ExistingType`**
- Do **not** introduce new types at runtime
- Compiler expands aliases to underlying types

Common Use Cases

- Shorten long generic types
- Name function types for readability
- Shorten inner/nested class references
- Can be nested inside classes (Kotlin 2.x)

⚠ Not type-safe!

- Alias and original type are interchangeable
- Use value classes for real type safety

Practice Exercises

Exercise 1: Array Operations

Create an IntArray of 5 squares ($0^2..4^2$), print the sum, and convert it to a List. Then check if 9 is in the array.

Solution

```
fun main() {
    val squares = IntArray(5) { it * it }

    println(squares.joinToString())
    println("Sum: ${squares.sum()}")

    val list = squares.toList()
    println("List: $list")
    println("9 in array: ${9 in squares}")
}
```

Concepts Used:

- IntArray constructor with lambda
- .sum() on primitive arrays
- .toList() conversion
- in operator for containment check

```
Output:
0, 1, 4, 9, 16
Sum: 30
List: [0, 1, 4, 9, 16]
9 in array: true
```

Exercise 2: Smart Casts & when

Write a function that takes an Any parameter and uses when + is to return a description: double the Int, uppercase the String, or "Unknown" for other types.

Solution

```
fun transform(input: Any): String =
    when (input) {
        is Int    -> "Doubled: ${input * 2}"
        is String -> input.uppercase()
        else      -> "Unknown"
    }

fun main() {
    println(transform(21))
    println(transform("hello"))
}
```

Concepts Used:

- when expression with is checks
- Smart cast to Int (input * 2)
- Smart cast to String (.uppercase())
- Any type as parameter

```
Output:
Doubled: 42
HELLO
```

Exercise 3: Type Alias & Safe Cast

Define a typealias for a predicate function. Use it with safe cast (as?) to filter an array of mixed Any values.

Solution

```
typealias Predicate<T> = (T) -> Boolean

fun main() {
    val mixed : List<Any> = listOf(1, "two", 3, "four", 5)

    // Define predicate for even numbers
    val isEven: Predicate<Int> = { it % 2 == 0 }

    // Extract ints using safe cast (as?) and filter nulls
    val ints = mixed.mapNotNull { it as? Int }
    val evens = ints.filter(isEven)

    println("Original: $mixed")
    println("Ints: $ints")
    println("Evens: $evens")
}
```

Concepts Used:

- typealias Predicate<T> for function type
- Safe cast (as?) to filter non-Int values
- mapNotNull + safe cast pattern
- Passing alias as filter predicate

Output:

```
Original: [1, two, 3, four, 5]
Ints: [1, 3, 5]
Evens: []
```

Your Turn — Practice Exercises

1 Array Builder

Create arrays of 10 elements using: arrayOf, Array constructor, and intArrayOf. Compare them with contentEquals().

Hint: contentEquals() compares element-by-element.

2 Type Detective

Write a function that takes a List<Any> and prints the count of each type (Int, String, Double, other).

Hint: Use is checks inside a when block.

3 Safe Cast Chain

Given an Any? value, safely cast to String, then to Int, handling null at each step. Print the result or "Invalid".

Hint: Chain as? with ?.let { } or ?:

4 Alias Refactor

Refactor code with long generic types (Map<String, List<Pair<Int, String>>>) using type aliases. Verify the alias and original type are interchangeable.

Hint: Aliases don't create new types.

Challenge Exercises

5 Polymorphic Array Processor

Create an Array<Any> with mixed types. Use is checks to sum all Ints, concatenate all Strings, and count all other types. Print a summary report.

Hint: Combine arrays, smart casts, and type aliases.

6 DSL with Type Aliases

Design a mini event system using type aliases for event handlers: typealias EventHandler<T> = (T) -> Unit. Register and fire events with typed handlers.

Hint: Combine generics, type aliases & function types.

Tips for Success

- Experiment in the Kotlin Playground (play.kotlinlang.org)
- Use arrays for performance-critical code, collections for everything else
- Smart casts are your friend — prefer is checks over explicit as casts
- Type aliases improve readability but don't provide type safety — consider value classes
- Read the official docs for edge cases and advanced patterns